

Lecture 4: Equilibrium — Aiyagari

第四讲：均衡 — Aiyagari

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 14, 2026

University of Toronto

Course Overview / 课程概览

Course Overview

课程概览

- Week 1: **Stages**
- Week 2: **Outer Loops**

Heterogeneous-agent macro models are made by combining **Stages** with **Outer Loops**.

By showing you how to use **Stages** and **Outer Loops**, I can show you how to solve **many, many models**.

- 第一周: **Stages**
- 第二周: **外层循环 (Outer Loops)**

异质性主体宏观模型由 **Stage** 与**外层循环** 组合而成。

只要教会你使用 **Stage** 与**外层循环**，我就能带你求解**许许多多的模型**。

Stages

Stages

The primary purpose of this course is to introduce you to solving heterogeneous-agent macro models using **Stages**.

Stages are the basic building blocks of heterogeneous-agent macro models. They define household problems, which are the **hard part**.

Because **Stages** are conceptually **well-defined**, **modular**, and **composable**, you can build complex macro models by simply combining **Stages**.

本课程的主要目的，是教你用 **Stage** 来求解异质性主体宏观模型。

Stage 是异质性主体宏观模型的基本构件。它们定义家庭问题——也就是建模中**最难的部分**。

因为 **Stage** 在概念上**定义清晰**、**模块化**、**可组合**，只需把若干 **Stage** 拼起来，就能搭出复杂的宏观模型。

Stages Solve the Household Block

Stage 解决家庭模块

A **Stage** defines how households make decisions given their **environment**: **prices** and **parameters**.

We can combine **Stages** to solve the **household block**: Value functions and populations given prices and parameters. This allows us to solve for **aggregates**, such as:

- Goods demand
- Capital demand
- Labour supply

一个 **Stage** 定义了给定环境（价格与参数）下，家庭如何作决策。

把若干 **Stage** 组合起来，就能解出**家庭模块**：在给定价格与参数时的价值函数与人口分布。

由此即可求出各类**总量**，例如：

- 商品需求
- 资本需求
- 劳动供给

Everything Else is Outer Loops

其余的一切都是外层循环

If we solve the **household block** using **Stages**, everything else is **Outer Loops**.

Equilibrium: Solve **market clearing condition** in an outer loop, together with **firm** or **government** blocks.

Steady State: Iterate population λ forward.

Transition: Solve for a **path** of market-clearing prices.

Global Solution with Aggregate

Uncertainty: Solve for value function conditional on aggregate state.

当我们用 **Stage** 解了**家庭模块**之后，余下的一切都是**外层循环**。

均衡：在外层循环里求解**市场出清条件**，并与**企业**或**政府**模块联立。

稳态：把人口 λ 向前迭代。

过渡：求出市场出清价格的一条**路径**。

含聚合不确定性的全局解：在给定聚合状态下求价值函数。

Why Stages? 为何要用 Stage?

Stages are **composable**: You can build complex models **simply** by combining Stages.

Stages have **precise mathematical definitions**: You can define and solve a model without worrying about **implementation details**.

Stages are **standardized**: You can **download** optimized implementations of Stages, and use them to build **your models**. (e.g. My HouseholdStages.jl, the SSJ Toolkit, or ECON-ARK).

Stage 是**可组合的**: 把若干 Stage 拼起来, 就能**轻易地**搭出复杂模型。

Stage 有**精确的数学定义**: 在定义并求解模型时, 无需担心**实现细节**。

Stage 是**标准化的**: 你可以**下载**经过优化的 Stage 实现, 用它们来搭建**自己的模型**。(e.g. 我的 HouseholdStages.jl, SSJ Toolkit, 或者 ECON-ARK)

Understand Definition, not Implementation

理解定义，而非实现

I want to teach you what stages **are** and how you can **use** them.

My course notebooks provide **simple** implementations of stages, so you can understand **what they are**.

I also provide highly **optimized** implementations of stages. You do **not** need to understand how they work.

我要教你的是 Stage **是什么**，以及如何**使用**它们。

课程的 notebooks 给出 Stage 的**简单**实现，目的是让你理解它们**是什么**。

我也提供了高度**优化**过的 Stage 实现——但你**并不需要**理解它们是怎么工作的。

Analogy: Multiplication

类比：乘法

It is **useful** that computers can multiply numbers and matrices together.

You don't usually need to know **how** your computer does it: logic gates, binary numbers, floating point arithmetic, etc.

But you **do** need to know how multiplication is **defined**: what is the **precise mathematical definition** of matrix multiplication, for example.

计算机能把数与矩阵相乘——这件事非常**有用**。

你通常不必知道它**怎么**做到：逻辑门、二进制、浮点运算等等。

但你**确实**需要知道乘法是如何被**定义**的——例如矩阵乘法的**精确数学定义**。

Analogy: Cars

类比：汽车

Cars are manufactured using **standardized components**: Wheels, engines, chassis.
I will provide you with the standardized components of **heterogeneous-agent macro models**:

1. Stages
2. Outer Loops

汽车由**标准化部件**制造而成：轮子、发动机、底盘。
我将给你**异质性主体宏观模型**的标准化部件：

1. Stages
2. 外层循环 (Outer Loops)

How to Use This Material

如何使用这套材料

In the future, I hope you will use build your own models by combining **Stages** and **Outer Loops**.

I will not cover specific topics in depth, but you can study:

1. Saving
2. Inequality
3. Spatial MigrationStage
4. Housing Markets
5. Entrepreneurship
6. Climate Change
7. And much more!

将来，希望你能把 **Stage** 与**外层循环**组合起来，搭建**你自己的模型**。

我不会深入讨论某个具体主题，但你可以自行研究：

1. 储蓄
2. 不平等
3. 空间迁移
4. 房地产市场
5. 创业
6. 气候变化
7. 还有更多！

Today

今日

Today, we will transition from **defining Stages** to **using Stages**.

The first application: adding **equilibrium** and solving a complete **general equilibrium model**. (Aiyagari, 1994).

今天，我们要从**定义 Stage** 过渡到**使用 Stage**。

第一个应用：加入**均衡**，求解一个完整的**一般均衡模型**（Aiyagari, 1994）。

Aiyagari — The Model / Aiyagari 模型

Three Blocks 三个模块

Like many macro models, Aiyagari has three blocks:

1. A **household block** made of **Stages**.
2. A **firm** that turns capital and labour into goods.
3. A notion of **equilibrium** (steady state).

Familiar economics:

1. A demand side.
2. A supply side.
3. A notion of equilibrium.

与多数宏观模型一样，Aiyagari 由三个模块构成：

1. 由 **Stage** 搭成的**家庭模块**。
2. 把资本与劳动转化为商品的**企业**。
3. 一个**均衡**的概念（稳态）。

熟悉的经济学：

1. 需求侧。
2. 供给侧。
3. 均衡的概念。

Plan for the rest of the lecture

本讲剩余安排

1. Recall the household block from L3.
2. Read aggregates $\bar{K}$, $\bar{L}$, $\bar{C}$ off λ^* .
3. One-line firm: $Y = K^\alpha L^{1-\alpha}$.
4. Define equilibrium: $\bar{C} = Y$.
5. Solve for the equilibrium R in an **outer loop**.

Everything outside the household block is just plain code on top of what we already have.

1. 回顾第三讲的家庭模块。
2. 从 λ^* 读出总量 $\bar{K}$ 、 $\bar{L}$ 、 $\bar{C}$ 。
3. 一行代码的企业： $Y = K^\alpha L^{1-\alpha}$ 。
4. 给出均衡的定义： $\bar{C} = Y$ 。
5. 用一个**外层循环**求出均衡 R 。

除家庭模块外的一切，都只是在已有代码之上写几行外壳。

The Household Block / 家庭模块

We already built this
我们已经搭好了

In Lecture 3 we built a household block from three Stages:

1. Income shock (Markov on w).
2. Income (cash-on-hand $b = R b^{\text{end}} + w$).
3. Consumption–saving (grid search on b^{end}).

Backward sweep $\rightarrow$ value function V and policy c^* .

Forward sweep $\rightarrow$ stationary distribution λ^* .

Today: same code but R becomes a *price*

在第三讲中我们用三个 Stage 搭好了家庭模块：

1. 收入冲击 (w 上的马尔可夫)。
2. 收入 (组装现金 $b = Rb^{\text{end}} + w$)。
3. 消费-储蓄 (在 b^{end} 上做网格搜索)。

后向 $\rightarrow$ 价值函数 V 与策略 c^* 。

前向 $\rightarrow$ 平稳分布 λ^* 。

今天：同一份代码，但 R 变成需要求解的价格。

Solve the household block at a given R 在给定 R 下求解家庭模块

```
function solve_aiyagari_steady_state(params; verbosity=1)
    V_end = solve_vfi(params; verbosity)
           = find_steady_state_population(V_end, params; verbosity)
    return (; V_end, )
end
```

One call: VFI to convergence, then forward-iterate λ until it stops moving.

Output: V and λ^* .

Everything inside is the L3 code, unchanged.

一次调用：VFI 收敛后，再把 λ 前向迭代到不再变化。

输出 V 与 λ^* 。

里面用的全是第三讲那份代码，原样不动。

Aggregates / 总量

Read $\bar{K}, \bar{L}, \bar{C}$ off λ^*
从 λ^* 读出 $\bar{K}, \bar{L}, \bar{C}$

Every aggregate is an inner product against λ^* :

$$\bar{K} = \sum_{i,j} b_i \lambda_{ij}^*$$

$$\bar{L} = \sum_{i,j} \lambda_{ij}^*$$

$$\bar{C} = \sum_{i,j} c_{ij}^* \lambda_{ij}^*$$

$$\bar{V} = \sum_{i,j} v_{ij} \lambda_{ij}^*$$

Labour supply $\bar{L}$ is just the (un-normalized) population mass — exogenous

每个总量都是与 λ^* 的内积:

$$\bar{K} = \sum_{i,j} b_i \lambda_{ij}^*$$

$$\bar{L} = \sum_{i,j} \lambda_{ij}^*$$

$$\bar{C} = \sum_{i,j} c_{ij}^* \lambda_{ij}^*$$

$$\bar{V} = \sum_{i,j} V_{ij} \lambda_{ij}^*$$

劳动供给 L 就是（未归一化的）人口总质量——外生。

compute_household_aggregates
计算家庭总量

```
function compute_household_aggregates(;param_vals...)
    params = get_params(;param_vals...)
    (V_end, ) = solve_aiyagari_steady_state(params)

    b_grid_policy = policy_function(V_end, params)
    c_grid_policy = get_c_policy(params.b_grid, b_grid_policy)

    L_agg = sum( )
    K_agg = sum(params.b_grid .* )
    V_agg = sum(V_end .* )
    C_agg = sum(c_grid_policy .* )

    return (; V_end, , L_agg, K_agg, V_agg, C_agg)
end
```

Take parameter overrides (notably R), solve

接收参数覆盖 (重点是 R)，求解家庭模

10-minute break
休息 10 分钟

The Firm / 企业

Cobb–Douglas

Cobb–Douglas 生产函数

A representative firm with Cobb–Douglas production:

$$Y = K^{\alpha} L^{1-\alpha}.$$

Given aggregate K and L from the household block, output Y is a one-liner.

No firm dynamics, no entry, no heterogeneity — just a production function.

The entire supply side is one line of code.

代表性企业，Cobb–Douglas 生产函数：

$$Y = K^{\alpha} L^{1-\alpha}.$$

给定家庭模块给出的总量 K 与 L ，产出 Y 一行代码就算出来。

没有企业动态，没有进入退出，没有异质性——就是一个生产函数。

整个供给侧只有一行代码。

compute_Y 计算产出

```
compute_Y(K, L;  =1/3) = K^  * L^(1- )
```

That's it. Pass the household block's $\bar{K}$ and $\bar{L}$ in, get Y out.

就这么简单。把家庭模块的 $\bar{K}$ 与 $\bar{L}$ 传进来，就得到 Y 。

Equilibrium / 均衡

Goods market clearing 商品市场出清

In steady state, what households consume must equal what the firm produces:

$$\bar{C} = Y.$$

Define the *excess demand* for goods:

$$\text{excess_demand}(R) := \frac{\bar{C}(R) - Y(R)}{Y(R)}.$$

Equilibrium is the R that makes $\text{excess_demand}(R) = 0$.

稳态下，家庭消费必须等于企业产出：

$$\bar{C} = Y.$$

定义超额需求：

$$\text{excess_demand}(R) := \frac{\bar{C}(R) - Y(R)}{Y(R)}.$$

均衡即满足 $\text{excess_demand}(R) = 0$ 的 R 。

Why $\bar{C}$ and Y both depend on R

R flows through the household block:

$$R \rightarrow V, c^* \rightarrow \lambda^* \rightarrow (\bar{K}, \bar{C}).$$

Then through the firm:

$$(\bar{K}, \bar{L}) \rightarrow Y.$$

So one R pins down one $(\bar{C}, Y)$ pair. Move R until they agree.

This loop is exactly the **Outer Loop** we promised on day one.

R 沿家庭模块传递:

$$R \rightarrow V, c^* \rightarrow \lambda^* \rightarrow (\bar{K}, \bar{C}).$$

再交给企业：

$$(\bar{K}, \bar{L}) \rightarrow Y.$$

所以每个 R 唯一确定一对 $(\bar{C}, Y)$ 。挪动 R 直到二者一致。

这个循环正是第一天承诺的**外层循环**。

compute_excess_demand
超额需求

```
function compute_excess_demand(R)
    res = compute_household_aggregates(;R)
    Y_agg = compute_Y(res.K_agg, res.L_agg)
    return excess_demand = (res.C_agg - Y_agg) / Y_agg
end
```

Solve the household block at this R , plug $(\bar{K}, \bar{L})$ into the production function, return the relative gap between $\bar{C}$ and Y .

The household block (Stages) and the firm (one line) are glued together by this little wrapper.

在给定 R 下解家庭模块，把 $(\bar{K}, \bar{L})$ 代入生产函数，再返回 $\bar{C}$ 与 Y 的相对差距。

家庭模块（Stage）与企业（一行）就靠这个小包装函数粘在一起。

Solve for the equilibrium R

求解均衡 R

```
tol = 0.0001
lr  = 0.01

R    = 1.03
err = Inf
while err > tol
    excess_demand = compute_excess_demand(R)
    R = R + excess_demand * lr
    println("R=$R\t excess_demand=$excess_demand")
end
```

A simple gradient-style update: nudge R in the direction of excess goods demand.

If $\bar{C} > Y$, households are consuming more than the firm produces — raise R to

一个简单的梯度式更新：按超额需求的符号微调 R 。

若 $\bar{C} > Y$ ，家庭消费多于企业产出——提高 R 鼓励储蓄，使 K （进而 Y ）变大。

Wrap / 收尾

What we built today

今天搭出了什么

A complete Aiyagari model:

- Household block: three **Stages** from L3, reused unchanged.
- Firm: one line of code.
- Equilibrium: $\bar{C} = Y$.
- An **Outer Loop** on R to enforce it.

Stages do the demand side. The outer loop closes the model.

Notice the asymmetry: the hard part (household block) is the part we get for free.

一个完整的 Aiyagari 模型:

- 家庭模块: 第三讲的三个 **Stage**, 原封不动。
- 企业: 一行代码。
- 均衡: $\bar{C} = Y$ 。
- 在 R 上的**外层循环**保证均衡成立。

Stage 负责需求侧; 外层循环把模型闭合。
注意这种不对称: 最难的部分 (家庭模块) 反而是“现成的”。

Next
接下来

The same template gives us many more models:

- Add depreciation, a government, taxes, transfers — modify only the firm/outer-loop layer.
- Add housing, entrepreneurship, durables — add another Stage.
- Replace gradient descent with bisection, Newton, or a continuation method — change only the outer loop.

Same picture every time:

同一套模板可生出许多模型：

- 加入折旧、政府、税收、转移支付——只改企业/外层循环。
- 加入住房、创业、耐用品——添一个 Stage。
- 把梯度更新换成二分、Newton 或同伦法——只改外层循环。

每次都是同一幅图：

Stages + 外层循环.

Open Q&A and live-coding 答疑与现场代码

tutorial_notebooks/L04_aiyagari.ipynb